1. Project Overview

The goal of this project was to build a secure web application called SecureShield. We used FastAPI for the backend and implemented several security layers to protect user data. The project covers password hashing, JWT authentication, and role-based access control.

2. Security Implementation

Password Storage (Task 1): We used the Bcrypt library to store passwords securely. Instead of saving real passwords, we save a "hash." We also used salting to make sure that even if two users have the same password, their hashes will look different.

Authentication (Task 2 & 3): When a user logs in with the correct credentials, the server creates a JSON Web Token (JWT). This token is sent to the user and must be included in the header of every request to prove their identity.

Access Control (Task 4): We created two roles: User and Admin. Regular users can see their profile, but only Admins are allowed to delete users. If a regular user tries to delete someone, they get a 403 Forbidden error.

Logging (Task 6): We set up a defensive logging system. Every time someone tries to access a restricted function (like a user trying to delete a record), the system writes a warning in the security.log file.

3. Theoretical Questions (Task 7)

Q1: Why is "Salting" important for password hashing? Salting is a technique where we add a random string of characters to a password before hashing it. This is important because it protects against Rainbow Table attacks. Without salt, a hacker could use a pre-computed list of common hashes to find passwords very quickly. With salt, every hash is unique, making it much harder to crack.

Q2: What are the security risks of storing sensitive data in a JWT? The main risk is that the data inside a JWT (the Payload) is only encoded (using Base64), but it is not encrypted. This means anyone who gets the token can go to a site like jwt.io and read all the information inside, such as usernames or roles. Therefore, we should never store sensitive data like passwords or private phone numbers inside a JWT.

4. Security Validation (Tamper Test)

To test the system, I tried to "hack" my own token by changing my role from User to Admin on the jwt.io website. When I tried to use this modified token on my server, it returned a 401 Unauthorized error. This happened because the Signature was invalid. Since I don't have the server's SECRET_KEY, I cannot create a valid signature for a modified token.

5. Conclusion

The SecureShield project successfully demonstrates how to protect a web application from common attacks. By using hashing, tokens, and logging, we created a system that ensures only authorized people can access sensitive data.